

CapAge Implementation Architecture v0.1

Governance Plane and Trust Boundaries

CapAge's governance plane is a **strictly layered architecture**: the immutable *Constitution* (Articles 1–105) sits at the top, followed by approved **constitutional amendments**; below that, **owner policy** configures delegated choices; then explicit **authority grants** (capabilities) and **operating parameters** (budgets, risk limits, etc.); finally the agent's **strategy** and routine tasks. All constitutional text and amendments are stored in a write-once, versioned repository protected by cryptographic hashes [39†L39-L47] [51†L1377-L1384] . The Constitution (C) itself is hard-coded into this repository and only changable via the owner's explicit amendment process (C). Owner identity (and key) is held outside the agent's control (P): for example, in a hardware-backed wallet or vault. Owner-set policies (P) – like risk tolerance or domain scope – are stored in a signed, auditable policy store. Active **authority grants** (G) are kept as unforgeable tokens in an internal database (or distributed policy store) and consumed by the agent's orchestrator; the agent cannot modify or fake these tokens. **Operating parameters** (O) – e.g. monthly spend limits or concurrency caps – are likewise externally defined and read-only at runtime. An append-only *governance log* records all changes to these artifacts [39†L39-L47] : amendments, policy updates, grant issuances/revocations, and parameter changes, each chained by hashes. This log is write-once and cryptographically protected (e.g. via Merkle chaining [39†L39-L47] [39†L94-L100]), ensuring the agent cannot erase or rewrite history (C). Physically, these components run in a **separate, high-trust enclave or service**. The strategic agent process runs unprivileged, with **no direct API or filesystem access** to governance data [50†L951-L959] [51†L1377-L1384] . All agent requests for governance data or actions (e.g. reading policy, authorizing itself) must go through well-defined enforcement interfaces. In particular, the agent's reasoning engine is sandboxed from the validation/enforcement engine (C): the Constitution's rules are enforced by separate code that the agent cannot inspect or alter [50†L951-L959] . For example, the agent is prevented at the OS level (via container or process isolation) from reading the validation service's code or logs [50†L951-L959] [51†L1377-L1384] . This **trusted computing base** (the governance enclave plus ledger) is the universal reference for compliance. In short, any component or configuration outside this trust boundary – especially model weights, dynamic prompts, or strategy code – cannot change constitutional rules, owner identity, or audit history.

Authorization and Capability Model

All actions require an explicit, machine-verifiable authorization grant (C/G). We implement a **capability-based access control model** [18†L59-L67] [31†L126-L134] . At issuance time, the *Capability Service* encodes each grant as a signed token (e.g. a JWT or Macaroon [31†L126-L134]) with fields for: actor identity, action type, target resource (account, tool, counterparty), scope constraints (limits, time window, data class), delegation depth, and revocation ID. Tokens include an issuer signature and a short expiry [31†L126-L134] [31†L221-L226] . For example, a grant might allow “transfer up to \$100 to Company X” until a given timestamp. The orchestrator requests tokens by providing intent context (conversation ID, relevant accounts), and a policy engine (e.g. OPA/Cedar) checks owner policies and constitutional rules to produce a token [31†L236-L244] . The agent must attach a valid token to every privileged call: tool

invocation, fund transfer, or contract submission. The target service **verifies the token** (signature, expiry, jti not revoked, and that the requested action matches scope) before execution [31†L236-L244] [33†L301-L304] .

Capability tokens enforce **least privilege**: an agent can hold only those rights explicitly given, and cannot escalate or reuse them outside scope [31†L126-L134] [18†L96-L104] . By design, no action is possible without a token. This closes classic confused-deputy and prompt-injection gaps: even if a malicious prompt tries to exploit the agent, it cannot perform any action for which it lacks a token [18†L96-L104] [18†L112-L119] . (As PADISO puts it, “if the agent doesn’t hold the token for ‘send arbitrary email,’ it physically cannot execute” that action [18†L96-L104] .) Delegation is implemented by allowing tokens to carry a `parent_jti` or related claim [31†L228-L230] : a parent token can mint scoped child tokens (up to a preset depth), which is itself logged for audit. Token revocation is immediate (by blacklist of jti) and also ensured by short TTLs [31†L221-L226] . We define **access decision semantics** so that *denial-by-default* and *deny overrides allow* (i.e. any lack of permission leads to refusal). Operating parameters (O) and policy filters (P) automatically intersect with grants: e.g. a token may cap spending based on a monthly budget policy. Ambiguous or incomplete requests cause automatic denial or escalation. Multiple tokens can be combined by intersection (the agent must satisfy all relevant grants) but never union (no implicit privilege).

In practice, enforcement is moved as close to the action as possible: each tool or API requires a token directly. The orchestrator acts as a token requestor and injector, but does not make final authorization decisions – that is done by token verification at the PEP. This decentralized check (tool-side) makes bypass nearly impossible [31†L236-L244] . We accept that some risky semantics (e.g. contractual complexity) cannot be fully checked by code, so any questionable token request triggers human (owner) review (C). Technical logs note any tokens presented, allowing us to catch TOCTOU or replay issues [33†L315-L323] . Transaction splitting and concurrency are handled by requiring each external action (payment, contract) to be uniquely identified (idempotency key) so retries or duplicates are detected.

Action-Risk Evaluation and Limits

CapAge treats each proposed action by its **consequence profile**. The Constitution mandates a *Maximum Plausible Loss (MPL)* for any activity (C), and caps exposure accordingly. We encode risk attributes (financial exposure, liability, reversibility, legal uncertainty, privacy impact, etc.) for each action type in our schema. For financial risk, the system estimates worst-case loss (e.g. asset devaluation, penalty fines) and compares it to deployable capital before allowing execution. This MPL calculation is updated with new data (market rates, legal analysis, past outcomes) and conservatively aggregated across correlated exposures. Whenever the MPL is unbounded or exceeds a threshold (owner policy or constitutional limit), the action is disallowed or escalated (C). We follow Reapp’s advice to “conduct a structured review around the maximum plausible loss and the worst recoverable outcome” [28†L119-L128] , using it to set hard limits on transaction size, counterparty, and budget (O) [28†L129-L136] . For example, we might automatically refuse transactions that could bankrupt the agent or impose unlimited liability.

Downside cannot be truly unbounded: any proposal with uncertain tail risk triggers owner scrutiny. In low-confidence cases, we apply reversible or staged execution (e.g. partial escrow or manual checkpoint) to limit impact. If data is missing or inconsistent (settlement unclear, external invoice delayed), we mark the related exposure in the ledger as provisional and prevent new actions until reconciled. In short, each action is

hypothesis-tested against worst-case impact, and only minimally privileged authority is granted in proportion to proven safety [28†L119-L128] [18†L112-L119] .

Economic and Accounting System

The heart of CapAge's economic layer is an **immutable ledger** recording every financial event, following event-sourcing principles [24†L120-L129] [39†L39-L47] . We maintain separate accounts for: **cash capital, owner equity, subsidies/investments, encumbered capital** (committed but unpaid), and **obligations/liabilities**. The ledger entries are *quadruple-entry*: every transaction updates four perspectives to stay balanced [45†L92-L100] . Concretely, a completed trade creates debits and credits for the agent's cash, the counterparty's account, and corresponding entries on the agent's internal profit/loss book. This follows the ABS's "four-entry accounting" model [45†L92-L100] to record transactions once in the agent's own accounts and once in the counterparty's (ensuring complete consistency).

Financial events (e.g. authorization, payment, refund, fee) are appended to the audit log [24†L120-L129] [39†L39-L47] . Obligations (commitments) are booked when an action is authorized, and actual flows (cash in/out) are recorded on settlement. Prepayments or deferred revenue are tracked separately, and any subsidy or grant (from owner) is tracked as non-contractual income with its own ledger tag (to prevent the agent from misclassifying owner loans as profits). We maintain both *cash accounting* (actual money movements) and *economic accounting* (including booked but unsettled items). Each event is signed and chained; tamper-evidence is enforced via cryptographic hashes on the ledger entries [39†L39-L47] [39†L94-L100] , so the agent cannot alter history. Projections (e.g. current cash balance, deployable capital) are derived by replaying this log [24†L120-L129] .

In practice, the system might use a transactional event store or blockchain-like log: either a centralized append-only database with multi-row transaction support, or a distributed ledger with consensus and Merkle proofs [39†L39-L47] . ACID-like semantics (via two-phase commits) ensure that each authorization, execution, and ledger-write occur atomically. If an external payment succeeds but the internal log crashes, we reconcile on recovery using unique transaction IDs and confirm with the payment rail (treating the log as source-of-truth [28†L119-L128]). Customer funds and third-party obligations are tagged distinctly (to avoid mixing owner capital with client assets). By construction, we can trace *owner capital* → *authorization* → *external action* → *liability* → *revenue* through the ledger. The Constitution's accounting concepts (QER, PEP, QEP etc.) are implemented via this double/quadruple-entry ledger, ensuring every credit has a corresponding debit (C) [45†L92-L100] . Results (income, losses) feed into the agent's net economic capital, subject to the owner's ability to cover deficits.

Evidence, Provenance, and Epistemic State

We separate **facts** from **hypotheses**. All inputs, observations, and decision artifacts include metadata on source and confidence. Structured decision records capture "what action was chosen, based on what evidence and rationale." For example, if the agent analyzes a contract, it logs the source file (with a content hash) and the LLM prompt/response pair that led to interpreting its terms. These records live in the append-only audit log [24†L120-L129] [51†L1377-L1384] , linking every decision to its evidence. The Constitution forbids hidden chains of reasoning, so we do *not* store raw chain-of-thought. Instead, we store distilled reasoning: key facts retrieved, relevant model outputs (summarized), and any external references (URLs,

docs) along with cryptographic fingerprints. This enables forensic reconstruction: by replaying the log and referencing source materials, an auditor can re-derive the conclusions.

Provenance is enforced via tagging: every piece of data or inference step carries provenance tags. When the agent ingests external content (news, APIs, vendor docs), that content is labeled with origin and trust level. Derived estimates (e.g. market forecasts) include their generation method (model used, data cutoff) and uncertainty bounds. Crucial decisions (e.g. risk assessment) are logged with their input evidence and the model/tool chain that produced them. For legal or financial validation, signed references to owner communications (approval emails, documented instructions) are recorded. In short, we treat the audit log as the unforgeable record of all “known truth” and how we reached decisions, enabling retrospective verification without needing full token traces [24†L120-L129] [39†L39-L47] .

Agent Runtime and Strategic Autonomy

The strategic core runs as a **neuromorphic agent** with a two-level memory system. **Working memory** (short-term context) resides in the LLM’s prompt window, holding the current task and conversation history. **Long-term memory** is external (e.g. a vector DB or knowledge store) for facts, lessons, and policies [30†L137-L141] . We classify internal state into *goals*, *hypotheses*, and *plans*. The agent can freely adapt its strategy (subject to constitutional guardrails) – it may generate new hypotheses and pursue novel opportunities – but it cannot override the governance layer. For example, it can autonomously choose to pivot to a new product idea, but before spending money it still must fetch an authorization token as above.

Memory and context interactions are carefully controlled. The agent may be allowed to *query* its memory or knowledge store via retrieval but may not write unvetted information into it; instead, new lessons become part of memory only after human review or cryptographic signing by a trusted submodule (D). Chain-of-thought reasoning remains internal and ephemeral. **Strategic state** (current objectives, high-level plan) is stored in a structured state machine that is updated only by sanctioned operations. Model prompts and external content are kept distinct from any governance symbols: for instance, even if a retrieved document mentions a policy term, it is treated as data not authority. Under the hood, we may mirror the “Agentic Event Sourcing” pattern [24†L145-L153] : the agent emits **intents** (proposed actions or knowledge updates), a deterministic orchestrator validates them against policy and ledger state, and only accepted intents are appended to the log. This ensures *intent vs execution separation*: the agent can imagine and suggest actions, but no actual state is mutated until the validation step.

Importantly, opportunity discovery is open-ended (D). We do not bake in any particular market-analysis funnel or scoring system into the governance plane. Instead, the agent is given tools for research and planning (e.g. web search, data analysis) under capability constraints [18†L59-L67] . It must still formulate its own experiments and hypotheses. All its outputs that aim to change the world are then checked against the constitutional rules. This preserves true autonomy (subject to (C)/(G) limits): CapAge can self-prioritize tasks, select models and vendors (D), and adapt workflows, as long as every real-world action stems from a valid grant.

Tooling, Delegation, and External-Input Security

CapAge uses **fine-grained tools** (APIs, sandboxes, models) under strict capability mediation [18†L59-L67] [31†L236-L244] . Every tool invocation by the agent must present a token with exactly the permissions

needed, as discussed above. The agent’s tool suite is minimized by need: per Parallax, the agent does not see all tools at once but only a “load_tools” meta-tool or specific group it requires [50†L852-L863]

[51†L1359-1364] . In practice, we load only tools relevant to the current task (“Text Search”, “Email API”, “CAD Simulator”, etc.), blocking any unknown tool loads. This *dynamic tool loading* enforces least privilege over time: even if compromised, the agent only has access to the tools it currently knows about [50†L861-L863] [51†L1359-L1364] .

Data flowing in from outside (documents, webpages, API responses) is treated strictly as **data inputs**. The agent never treats external text as policy or code. To mitigate indirect prompt injection, we rely on the fact that *no action occurs without validation*. Even if malicious content steers the agent’s reasoning, any resulting action must present a valid token, and will be re-checked in context [18†L96-L104] [50†L861-L863] . We may also employ static sandboxing: when the agent spawns a subordinate execution (e.g. runs Python code or a shell tool), it runs inside a microVM or container with no internet egress except whitelisted endpoints [33†L301-L304] . Secrets (tokens, credentials) are passed via secure channels only. Output from all tools is sanitized: e.g. any URL fetched is validated to ensure it’s on an allowlist, and any file the agent manipulates is sandboxed. The architecture enforces **information flow control** so that sensitive outputs (e.g. customer PII) cannot leak to low-trust sinks, even if the agent tries [50†L882-L889] .

Sub-agents (child LLMs) are strictly controlled delegates. The engine process mediates all inter-agent messaging and validation [51†L1351-L1359] . A sub-agent is given no discovery interface: it cannot enumerate or load any capabilities beyond what the parent explicitly passed [51†L1359-L1364] . In effect, the parent “inlines” a toolset to the child and trusts the orchestrator to enforce the same policy checks. All sub-agent outputs loop back through the same Shield as the main agent. This prevents delegation attacks or capability laundering: a rogue sub-agent cannot magically create new authority.

Owner Interaction and Approvals

The owner controls CapAge by reviewing and granting **specific authority requests**, not by micromanaging strategy. When CapAge needs powers beyond its current grants (e.g. a large funding request, a new market entry), it generates an **approval request record**. This record is presented via a secure interface (web UI or signed message) and includes: the exact action plan, economic rationale, alternatives considered, risk profile (including MPL), time urgency, affected accounts, and *precisely* the minimal token scope it needs. For example: “Execute purchase order #1234 for \$20,000 of widgets from Supplier Y; requested token: pay_supplier(USD 20k, acc-#456, 2026-12-31)”. The owner can review all details (with references to evidence) and either approve or reject.

We separate governance-level approvals (granting authority) from any strategic advice. The owner is not asked to design the strategy, only to check that CapAge has not asked for excessive powers. Declining a request (P) returns control to the agent with a logged refusal; the agent may revise and resubmit a narrower request if justified. We guard against *approval fatigue* by bundling routine recurring approvals into policy (with owner-set thresholds) and requiring explicit only for unusual cases. If owner contact is lost, CapAge automatically enters a safhold: it stops initiating new high-risk actions, freezes on minor ones, and enters a notify-only mode. This ensures owner control can always be asserted (C).

Security, Containment, and Shutdown

We assume a strong adversary (even a fully compromised agent). Using our layered model, we define defenses and fail-safes for each threat:

- **Governance corruption:** Constitution, grants, ledger run in a hardened enclave. Only owner-held keys can mutate them (C). All governance updates require multistep approval. Tamper-evident logs [39†L39-L47] [51†L1377-L1384] ensure any unauthorized modification is detected.
- **Credential compromise:** Capability tokens are short-lived (≤ 2 min) and tied to session IDs. If a token or agent key is stolen, we immediately revoke it (via quick blacklist) and panic-freeze any associated actions [28†L90-L98] . We also employ mutual authentication: commands from CapAge (e.g. to transfer funds) are signed by its active key, which can be rotated. Owner's revocation of CapAge's signing key forces a global pause (C).
- **Tool/database compromise:** Tools run in minimal containers with no external connectivity except approved endpoints [33†L301-L304] . The database storing governance state is accessible only through the enforcement service. If a tool crashes mid-action (e.g. after payment but before logging), reconciliation logic examines unique transaction IDs to avoid duplicates [28†L50-L59] . Data stores (leader/follower replicas) use cryptographic signatures on entries to detect tampering [39†L39-L47] .
- **Malicious input:** All external inputs are sanitized. For instance, HTML from web tools is stripped of scripts; binary files are quarantined. Because no action bypasses the shield, even steganographic payloads cannot force a forbidden outcome. The agent's own outputs (e.g. email text) are scanned for policy violations before release.
- **Failure during external transaction:** We use idempotency keys and two-phase commits. If the agent's process fails after sending payment but before logging, recovery checks the payment network (treat external network state as ultimate source of truth) and marks the ledger accordingly [28†L54-L62] . Refunds and chargebacks are handled via corresponding compensating entries in the ledger with audit trails.
- **Containment/Shutdown:** The owner can trigger an emergency **kill switch** (C) – implemented as an ingress rule or flag – that immediately halts all agent activity. This is an out-of-band control (e.g. cutting its network, revoking its keys). A verified kill command blocks even pending automated retries. Upon containment, the system preserves all in-flight logs and credentials. For partial failures, we follow Reapp's guidance: first freeze the affected capability or sub-agent, then coordinate recovery (e.g. query banks for actual balances) [28†L88-L96] [28†L106-L114] . No incident ever involves rewriting history: we always append a correction entry rather than erase a mistaken event [28†L106-L114] .

Residual risk is documented in a **failure mode catalogue**. We test these edges continuously (automated audits) and refine as needed. In summary, agent execution can be decisively stopped or rolled back at any time by owner or system logic, without trust in the agent itself [28†L119-L128] [50†L915-L923] .

Self-Modification and Deployment Lifecycle

CapAge may update its non-governance software (e.g. strategy code, ML models, data schemas) autonomously, but only within limits. We classify changes into two categories: *governance-affecting* vs *local*. Governance-affecting changes (which alter constitutional interpretation, ledger format, enforcement logic, owner controls, etc.) require owner approval (C). The system auto-analyzes any proposed update: if it

touches core modules or data definitions, it's flagged. For example, a change to the authorization logic or the event schema is parked for owner review (similar to a pull request). Non-governance changes (improving market models, swapping a tool, tuning parameters) can be done by CapAge with audit logging (D), as long as they do not alter how the constitutional rules run. All code changes go through a build pipeline that enforces static checks (e.g. "no calls to disable logging") and captures their provenance.

During deployment, we always perform a safe *canary test*: apply the update in a sandboxed instance with synthetic data to confirm it does not violate any invariant (e.g. does not nullify permissions). Rollback is possible by referencing the last trusted log checkpoint [24†L120-L129] . Backups of the governance database and ledger are kept off-agent and encrypted. In summary: CapAge can learn and evolve its business logic, but it cannot stealthily rewrite its constitution, nor can it upgrade itself out of owner control (C).

Constitutional Traceability and Issues

We have systematically mapped each constitutional clause to an architectural control (see Appendix). Every Article imposing a rule (C) has a direct enforcement mechanism: e.g. Articles on audit log become the append-only ledger [39†L39-L47] , Articles on limited spending map to capability tokens and risk checks [28†L119-L128] , Articles on owner veto become the kill switch control [50†L951-L959] , etc. Clauses delegating to owner policy (P) are honored by reading owner-configured parameters. No provision was found to be inherently self-contradictory or impossible under this design. In particular, we identify **no constitutional defects**: each requirement can be implemented faithfully via the above controls. For instance, concerns about the agent autonomously expanding its authority are addressed by strict capability enforcement and out-of-band owner interventions [51†L1377-L1384] [50†L951-L959] . Conversely, no suggested parameter or workflow (O/R) has been mistakenly frozen into law: all thresholds and heuristics here remain (O) adjustable and (R) replaceable.

Conclusion

CapAge v0.1's architecture combines a hardened governance kernel with a flexible agent core. The **governance plane** (constitution, policies, ledger) is an isolated, cryptographically protected domain. The **agent plane** (LLMs, models, subroutines) is empowered to explore and reason, but all real effects pass through enforceable tokens and checks [18†L59-L67] [50†L951-L959] . By leveraging capability-based permissions [18†L59-L67] [31†L126-L134] , event-sourced accounting [24†L120-L129] [39†L39-L47] , and strict isolation [50†L951-L959] [51†L1377-L1384] , the system ensures constitutional fidelity while allowing true entrepreneurial autonomy. Every component and data flow has an unbroken chain of custody, making the system auditable and secure. In practice, this means a team can implement CapAge v0.1 using existing tools (containerization, JWT/macaroon libraries, event stores, etc.) with confidence that no path exists for the agent to rewrite its own rules or spend beyond its mandate.

Sources: Key principles are drawn from best practices in agent security and immutable logging [18†L59-L67] [24†L120-L129] [28†L119-L128] [39†L39-L47] [50†L951-L959] [51†L1377-L1384] , adapted to satisfy all CapAge constitutional constraints.